Advanced Cloud Computing - Exam report

Agnese Valentini

November 2, 2026

1 Exercise 1: Network Performance Testing with Linux Network Namespaces

1.1 Setup for one veth pair

I chose to create the namespaces in my host machine with Ubuntu 24.04.1 LTS. I run these commands to create two network namespaces called ns1 and ns2

```
sudo ip netns add ns1
sudo ip netns add ns2
```

then I created a cable

```
sudo ip link add veth0 type veth peer name veth1
```

and I attached it to the namespaces

```
sudo ip link set veth0 netns ns1
sudo ip link set veth1 netns ns2
```

Now we can provide IP address to each interface

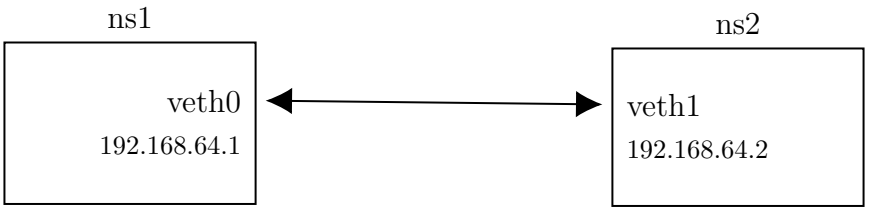
```
sudo ip -n ns1 addr add 192.168.64.1/24 dev veth0
sudo ip -n ns2 addr add 192.168.64.2/24 dev veth1
```

and eventually we can bring up the interface on each namespace

```
sudo ip -n ns1 link set veth0 up
sudo ip -n ns2 link set veth1 up
```

To test the connectivity between the namespaces and specifically the latency, it is useful to do ping

```
sudo ip netns exec ns1 ping 192.168.64.2
sudo ip netns exec ns2 ping 192.168.64.1
```



1.2 Setup for one veth pair with bridge

At first I created and activated the bridge br0

```
sudo ip link add br0 type bridge
sudo ip link set br0 up
```

Then I kept intact the previous veth pair, and I created a new one to be connected to the bridge

```
sudo ip link add veth2 type veth peer name veth3
sudo ip link add veth4 type veth peer name veth5
```

I moved veth2 to ns1 and veth4 to ns2

```
sudo ip link set veth2 netns ns1
sudo ip link set veth4 netns ns2
```

Then I attached veth3 and veth5 to br0

```
sudo ip link set veth3 master br0
sudo ip link set veth5 master br0
```

and added the IPs

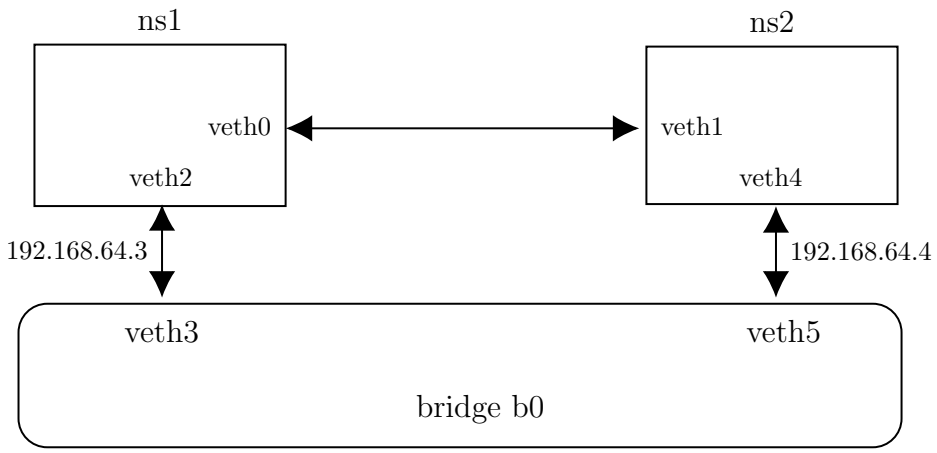
```
sudo ip -n ns1 addr add 192.168.64.3/24 dev veth2
sudo ip -n ns2 addr add 192.168.64.4/24 dev veth4
```

Eventually I brought up the new interfaces

```
sudo ip netns exec ns1 ip link set veth2 up
sudo ip netns exec ns2 ip link set veth4 up
sudo ip link set veth3 up
sudo ip link set veth5 up
sudo ip link set br0 up
```

and tested the connectivity through the bridge by running

```
sudo ip netns exec ns1 ping 192.168.64.4
sudo ip netns exec ns2 ping 192.168.64.3
```



1.3 Setup for two veth pairs, bridge and VXLAN

This network topology is the most complex between the three and it needs a bit of lines more. I took inspiration from a post in Thomas Maurice blog which is linked below. I began with creating another couple of pairs of virtual ethernet interfaces

```
sudo ip link add veth6 type veth peer name veth7
sudo ip link add veth8 type veth peer name veth9
```

Then, like I did in the previous sections, I moved each veth pair into the separate network namespaces

```
sudo ip link set veth6 netns ns1
sudo ip link set veth8 netns ns2
```

Then I created the two bridge interfaces called br1 and br2, set them up and connected them to the veth ends and also set them up

```
sudo ip link add name br1 type bridge
sudo ip link add name br2 type bridge
sudo ip link set br1 up
sudo ip link set br2 up
sudo ip link set veth7 master br1
sudo ip link set veth9 master br2
sudo ip link set veth7 up
sudo ip link set veth9 up
```

I also configured the IP addresses of the veths and set them up

```
sudo ip netns exec ns1 ip addr add 192.168.64.5/24 dev veth6
sudo ip netns exec ns1 ip link set veth6 up
sudo ip netns exec ns2 ip addr add 192.168.65.6/24 dev veth8
sudo ip netns exec ns2 ip link set veth8 up
```

Then I created two vxlan tunnel interfaces to connect the two namespaces, attached them to the two bridges and brought them up

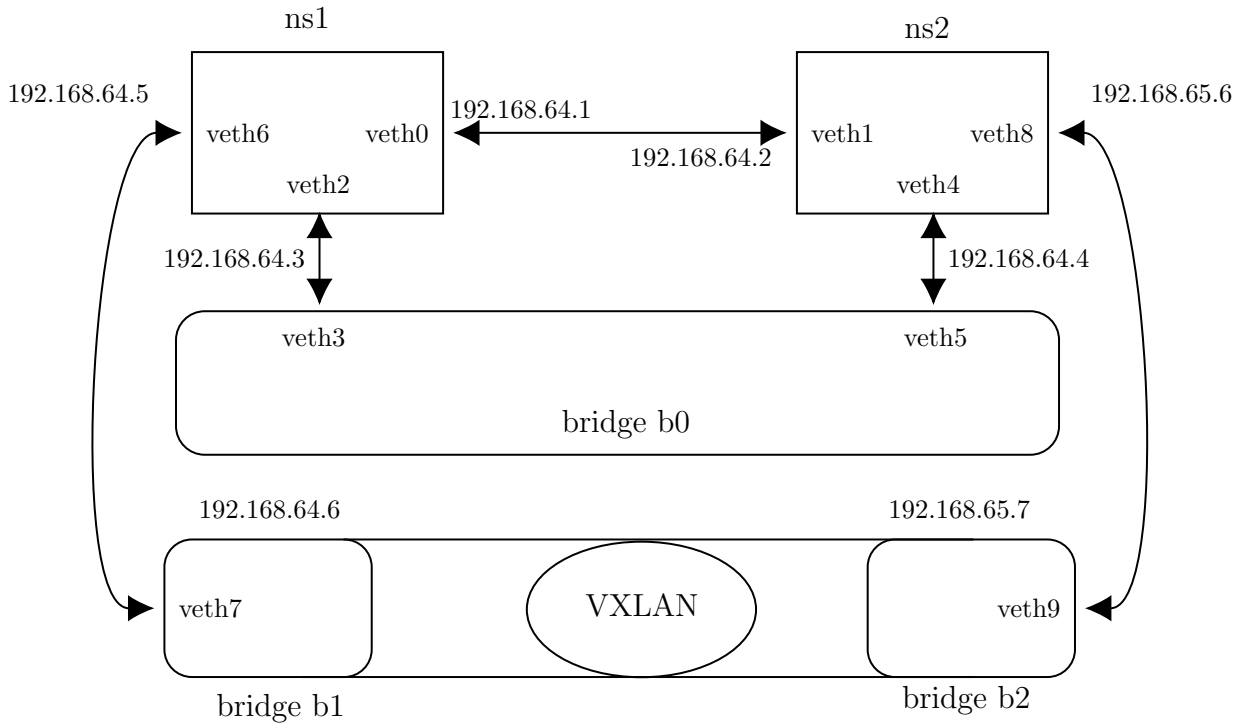
```
sudo ip link add vxlan1 type vxlan id 100 local 127.0.0.1 remote 127.0.0.1 dstport 4789 dev lo
sudo ip link add vxlan2 type vxlan id 101 local 127.0.0.1 remote 127.0.0.1 dstport 4790 dev lo
sudo ip link set vxlan1 master br1
sudo ip link set vxlan2 master br2
sudo ip link set vxlan1 up
sudo ip link set vxlan2 up
```

I added the route in both namespaces through the bridges and enabled ip forwarding

```
sudo ip addr add 192.168.64.6/24 dev br1
sudo ip netns exec ns1 ip route add 192.168.65.0/24 via 192.168.64.6
sudo ip addr add 192.168.65.7/24 dev br2
sudo ip netns exec ns2 ip route add 192.168.64.0/24 via 192.168.65.7
sudo sysctl -w net.ipv4.ip_forward=1
```

Eventually I checked if everything worked with ping

```
sudo ip netns exec ns1 ping 192.168.65.6
sudo ip netns exec ns2 ping 192.168.64.5
```



1.4 Performance testing

1.4.1 One veth pair

To test the bandwidth I used iperf3, after installing the package the command to run is

```
sudo ip netns exec ns1 iperf3 -s
```

in the ns1 that will act as a server and

```
sudo ip netns exec ns2 iperf3 -c 192.168.64.1
```

in the ns2 that will act as a client. Below the results

Namespace	Transfer	Bitrate
ns2 (sender)	68.8 GBytes	59.1 Gbits/sec
ns2 (receiver)	68.8 GBytes	59.1 Gbits/sec

Table 1: iperf3 results from ns1 to ns2

Then I tried to switch server and client using the same commands as before but changing the ns and ips and here are the results

Namespace	Transfer	Bitrate
ns1 (sender)	69.2 GBytes	59.4 Gbits/sec
ns1 (receiver)	69.2 GBytes	59.4 Gbits/sec

Table 2: iperf3 results from ns2 to ns1

To test the latency I used the mtr package that was already installed in my host machine. I run the command

```
sudo ip netns exec ns1 mtr --report --report-cycles 60 192.168.64.2
```

to test connectivity for 60 seconds and this is the result

HOST	Loss%	Snt	Last	Avg	Best	Wrst	StDev
ns2	0.0%	60	0.2	0.1	0.0	0.2	0.0

By running the command

```
sudo ip netns exec ns2 mtr --report --report-cycles 60 192.168.64.1
```

I obtained the exact same numbers as before in terms of loss and latency. For completeness here is the table

HOST	Loss%	Snt	Last	Avg	Best	Wrst	StDev
ns1	0.0%	60	0.2	0.1	0.0	0.2	0.0

1.4.2 One veth pair with bridge

Similarly as before, the commands to run iperf3 in this case are `sudo ip netns exec ns1 iperf3 -s` and `sudo ip netns exec ns2 iperf3 -c 192.168.64.3` and the other way round `sudo ip netns exec ns2 iperf3 -s` and `sudo ip netns exec ns1 iperf3 -c 192.168.64.4`. Results

Namespace	Transfer	Bitrate
ns2-br0 (sender)	62.8 GBytes	54.0 Gbits/sec
ns2-br0 (receiver)	62.8 GBytes	54.0 Gbits/sec

Namespace	Transfer	Bitrate
ns1-br0 (sender)	69.1 GBytes	59.4 Gbits/sec
ns1-br0 (receiver)	69.1 GBytes	59.4 Gbits/sec

Then the latency test with mrt with the commands

```
sudo ip netns exec ns1 mtr --report --report-cycles 60 192.168.64.4
```

HOST	Loss%	Snt	Last	Avg	Best	Wrst	StDev
ns2-br0	0.0%	60	0.1	0.1	0.1	0.2	0.0

```
sudo ip netns exec ns2 mtr --report --report-cycles 60 192.168.64.3
```

HOST	Loss%	Snt	Last	Avg	Best	Wrst	StDev
ns2-br0	0.0%	60	0.1	0.1	0.1	0.2	0.0

1.4.3 Two veth pairs, bridge and VXLAN

```
sudo ip netns exec ns1 iperf3 -s
sudo ip netns exec ns2 iperf3 -c 192.168.64.5
```

Namespace	Transfer	Bitrate
ns2-vxlan (sender)	44.1 GBytes	37.9 Gbits/sec
ns2-vxlan (receiver)	44.1 GBytes	37.9 Gbits/sec

```
sudo ip netns exec ns2 iperf3 -s
sudo ip netns exec ns1 iperf3 -c 192.168.64.6
```

Namespace	Transfer	Bitrate
ns1-vxlan (sender)	44.2 GBytes	38.0 Gbits/sec
ns1-vxlan (receiver)	44.2 GBytes	38.0 Gbits/sec

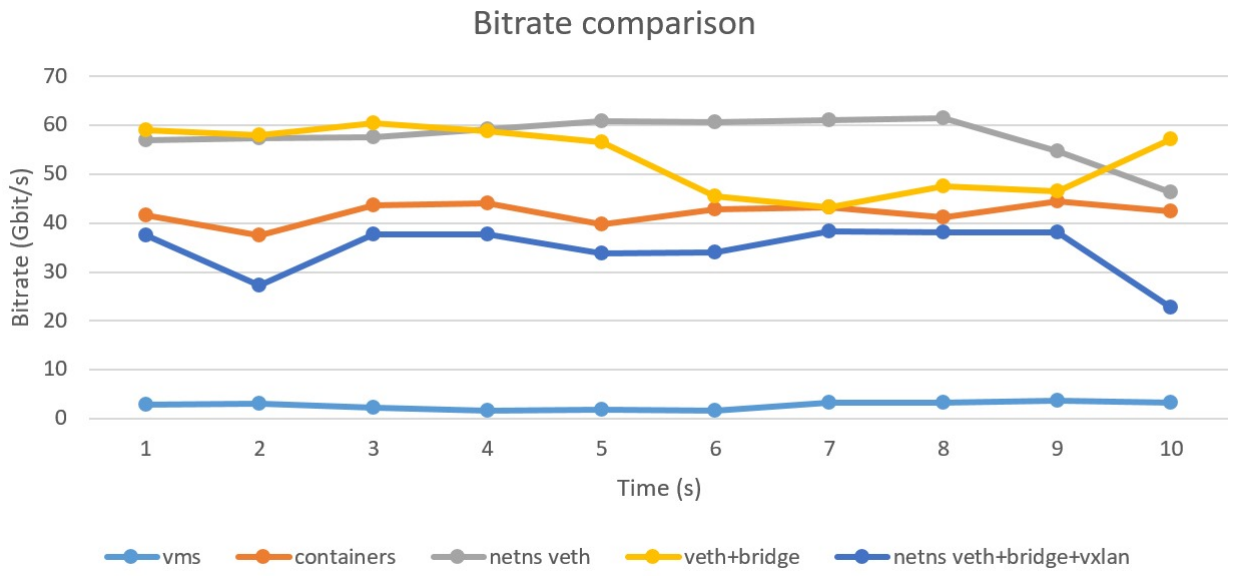
```
sudo ip netns exec ns1 mtr --report --report-cycles 60 192.168.64.6
```

HOST	Loss%	Snt	Last	Avg	Best	Wrst	StDev
ns2-vxlan	0.0%	60	0.1	0.2	0.0	0.3	0.1

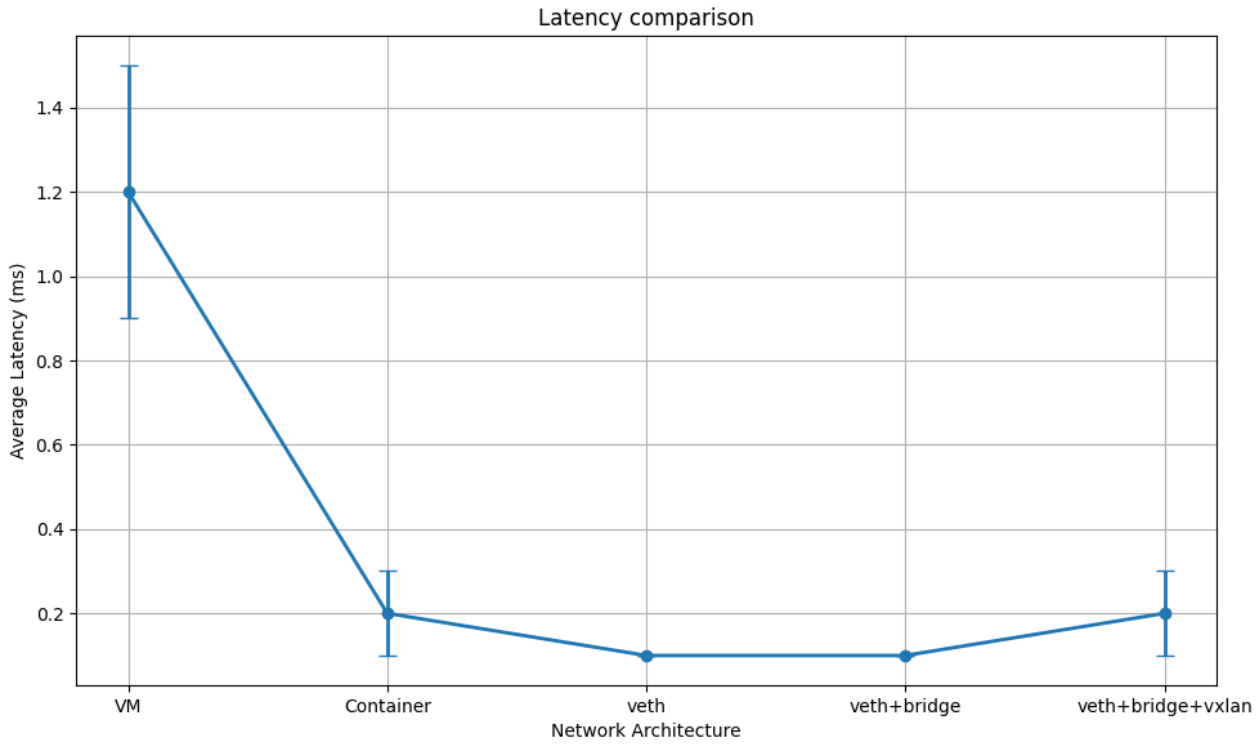
```
sudo ip netns exec ns2 mtr --report --reoirt-cycles 60 192.168.64.5
```

HOST	Loss%	Snt	Last	Avg	Best	Wrst	StDev
ns2-vxlan	0.0%	60	0.2	0.2	0.1	0.3	0.0

1.5 Comparison with Cloud Basic Case



This graph is made using iperf output in time in all different cases. It is clear from it that the vms have the worst performance, immediately followed by the most complex network topology in namespaces. Containers are slightly better than such topology but worse than the two "lighter" ones. This comes as expected, because of the layers that vm have and that containers do not, but we have also to keep in mind that all the network namenspaces were created on my host machine, so this could be why they perform better.



Also in this case, everything outperforms virtual machines which are not the most suitable choice if we need a small latency. Containers are similar to veth+bridge+vxlan as well as before, and the two best performances are the ones brought by veth and veth+bridge.

2 Exercise 2: Kubernetes Performance Monitoring with Prometheus

2.1 Setup

Installing Kubernetes is complicated, to say the least. So I followed the tutorial to install Kubelet on the documentation of Kubernetes and the tutorial on the page of CRI-O.

```
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gnupg
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.32/deb/Release.key |
sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
sudo chmod 644 /etc/apt/keyrings/kubernetes-apt-keyring.gpg

# installing cri-o
export CRIO_VERSION=v1.32
sudo curl -fsSL https://download.opensuse.org/repositories/isv:/cri-o:/stable:/$CRIO_VERSION/deb/Re
sudo  gpg --dearmor -o /etc/apt/keyrings/cri-o-apt-keyring.gpg
echo "deb [signed-by=/etc/apt/keyrings/cri-o-apt-keyring.gpg] https://download.opensuse.org/reposit
tee /etc/apt/sources.list.d/cri-o.list
sudo apt-get install cri-o kubeadm

sudo mv /etc/cni/net.d/10-crio-bridge.conflist.disabled /etc/cni/net.d/10-crio-bridge.conflist
sudo sed -i 's/10.85.0.0\/16/10.17.0.0\/16/' /etc/cni/net.d/10-crio-bridge.conflist

systemctl enable --now crio
systemctl enable --now kubelet # this command probably goes after kubeadm
systemctl status crio # to check the status of the service
sudo kubeadm config images pull --cri-socket=unix:///var/run/crio/crio.sock
sudo kubeadm init --pod-network-cidr=10.17.0.0/16 --cri-socket=unix:///var/run/crio/crio.sock

systemctl status kubelet # check kubelet status for debug reasons
```

```
journalctl -xeu kubelet # same

# initialization
sudo kubeadm init --pod-network-cidr=10.17.0.0/16 --cri-socket=unix:///var/run/crio/crio.sock

# activating the cluster
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# commands to interact with kubect (and thus with the pods and containers)
kubectl get pods
kubectl get services
kubectl describe service <my-service>
kubectl logs  <my-service>

# chose calico as CNI
kubectl create -f calico.yaml

ip addr add 10.110.3.215/24 dev eno1  # adding the ip of kubernetes
```

I run into the issue that after restarting the pc, kubelet lost the permission to read its configuration file. I think that this could be due to the fact that kubelet started before kubeadm, so I solved with chmod

```
cat /var/lib/kubelet/config.yaml
sudo chmod 777 /var/lib/kubelet/config.yaml
```

I added the image that I used for containers in Cloud basic to the Docker hub cloud

```
docker tag cloud_image:v1.0.0 agnesevalentini/cloud-advanced-exam:v1.0.0
docker push agnesevalentini/cloud-advanced-exam:v1.0.0
```

I installed Prometheus, Alertmanager and Grafana in Kubernetes as in the Github page of Prometheus

```
kubectl apply --server-side -f manifests/setup
kubectl wait \
--for condition=Established \
--all CustomResourceDefinition \
--namespace=monitoring
kubectl apply -f manifests/

# command to access prometeus dashboard
kubectl --namespace monitoring port-forward svc/prometheus-k8s 9090

# command to access grafana dashboard
kubectl --namespace monitoring port-forward svc/grafana 3000
```

The cluster is made of my computer that acts as a control plane node ad of another node that is the virtual machine called "master" and that was already created for the cloud basic case. I added it to the cluster using the command `kubeadm join` and generating a token to connect them. To check if they were correctly connected I run `kubectl get nodes -o wide`. My deployment has two pods with one container each, and each pod is contained in a node (host and master). They have the same resources as the cloud basic case. This is the yaml of the deployment:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cloud-adv-2pods
  labels:
    app: cloud-adv-2pods
spec:
  replicas: 2  # 2 pods with 1 containers each
  selector:
```

```
    matchLabels:
      app: cloud-adv-2pods
  template:
    metadata:
      labels:
        app: cloud-adv-2pods
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchLabels:
                app: cloud-adv-2pods
            topologyKey: "kubernetes.io/hostname"
      containers:
      - name: master
        image: agnesevalentini/cloud-advanced-exam:v2.0.0
        imagePullPolicy: Always
        resources:
          requests:
            memory: "1Gi"
            cpu: "1"
          limits:
            memory: "2Gi"
            cpu: "2"
        ports:
        - containerPort: 22
        securityContext:
          allowPrivilegeEscalation: true
          capabilities:
            add:
            - SYS_CHROOT
            - SETUID
            - SETGID
          runAsUser: 0
        env:
        - name: SSH_PORT
          value: "22"
        - name: TARGET_PORT
          value: "22"
        - name: HOST_IP
          valueFrom:
            fieldRef:
              fieldPath: status.hostIP
        - name: POD_IP
          valueFrom:
            fieldRef:
              fieldPath: status.podIP
```

I applied such deployment with the command

```
kubectl apply -f /home/agnese/magistrale/cloud_computing/advanced_exam_project/cloud_deployment_2po

# then I checked the name of the pods created and run
kubectl exec -it pod/<pod-name> -- bash
```

2.2 Results

To make the two nodes communicate I set up a host-only network following the Vbox documentation on the enp0s9 interface and I also connected them with SSH by running `ssh agnese@192.168.57.1` from vm to host and `ssh user@192.168.57.10` from host machine to VM. To check if they worked I simply added `bash` to the above commands and tried to do a `pwd`. This passage is essential to run the HPCC test. After this, I configured

the hpccinf.txt file with the same parameters as in the cloud basic case, so 10000 as Ns, 192 as NBs and 2 Ps and 2 Qs. To launch the HPCC tests, I run the command

```
mpirun -np 4 --host 10.17.19.144:2,10.17.219.76:2 hpcc
```

where the IPs are the ones of the two pods. These are the results that I got.

Metric	Value	Units
HPL (High-Performance Linpack)		
HPL_Tflops	0.0202718	Tflops
HPL_time	32.8938	seconds
HPL_eps	1.11022e-16	
DGEMM (Double-precision General Matrix Multiply)		
StarDGEMM_Gflops	7.30988	Gflops
SingleDGEMM_Gflops	6.79218	Gflops
PTRANS (Parallel Matrix Transpose)		
PTRANS_GBs	0.287137	GB/s
PTRANS_time	0.696531	seconds
PTRANS_residual	0	
RandomAccess		
MPIRandomAccess_LCG_GUPs	0.00220708	GUPs
MPIRandomAccess_LCG_time	80.9683	seconds
MPIRandomAccess_GUPs	0.0021086	GUPs
MPIRandomAccess_time	61.6172	seconds
STREAM		
StarSTREAM_Copy	2.97937	GB/s
SingleSTREAM_Copy	12.8778	GB/s
StarSTREAM_Add	2.43456	GB/s
SingleSTREAM_Add	9.37684	GB/s
FFT (Fast Fourier Transform)		
StarFFT_Gflops	1.19803	Gflops
SingleFFT_Gflops	1.71478	Gflops
MPIFFT_Gflops	1.32633	Gflops
MPIFFT_maxErr	1.91191e-15	

I attach the screenshot from Grafana dashboard made with the data recorded by Prometheus during the test (HPCC test performed on 7/5/25 between 11:20 and 11:40 UTC)

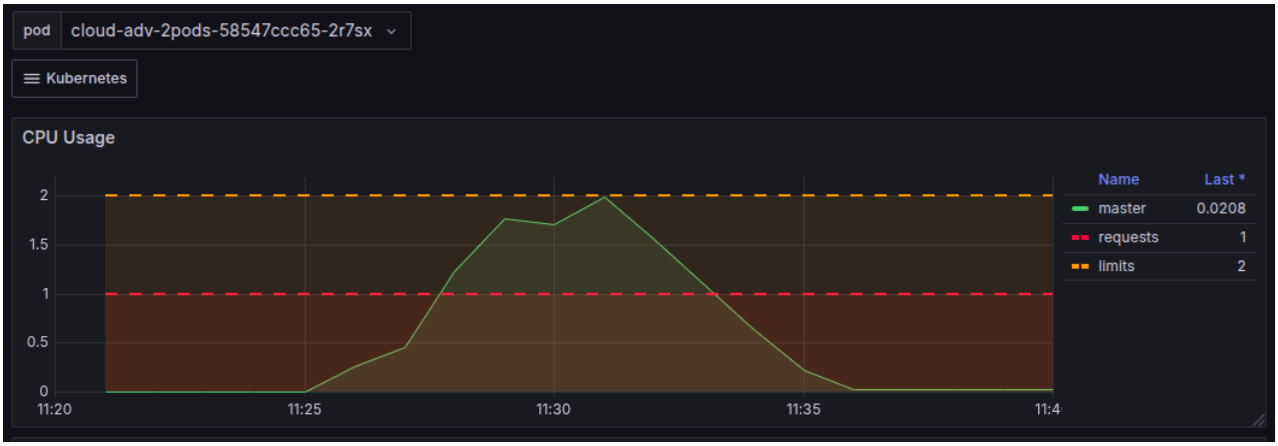


Figure 1: CPU usage pod on laptop

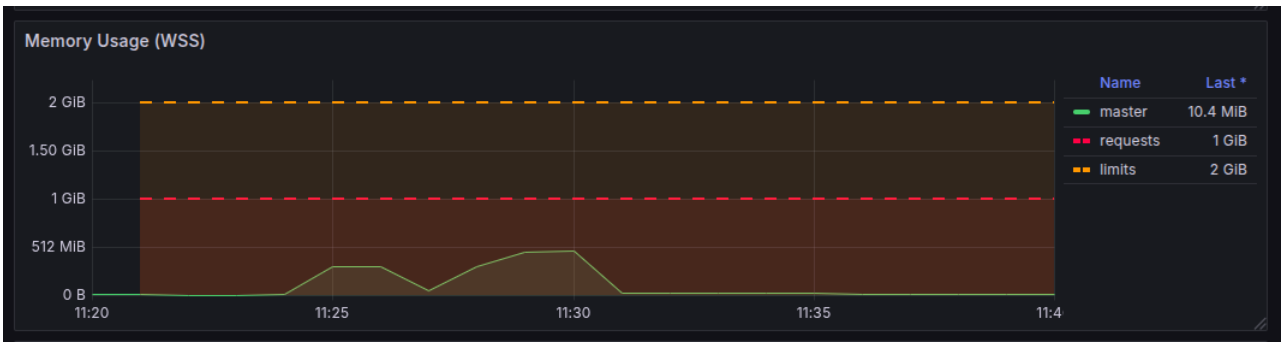


Figure 2: Memory usage pod on laptop

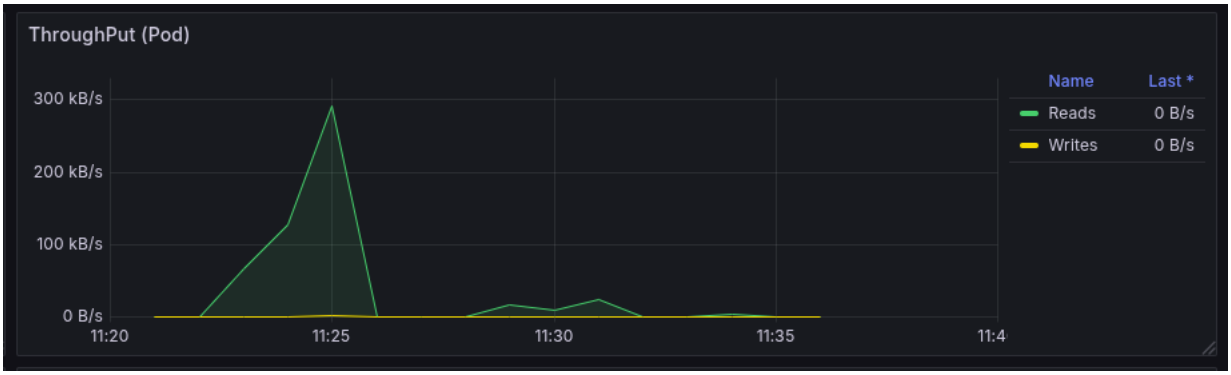


Figure 3: Network throughput pod on laptop

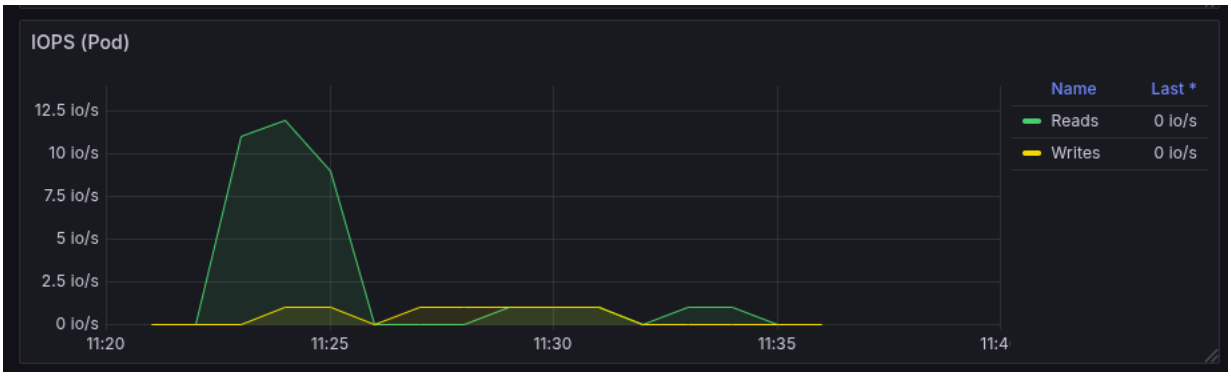


Figure 4: Disk I/O pod on laptop

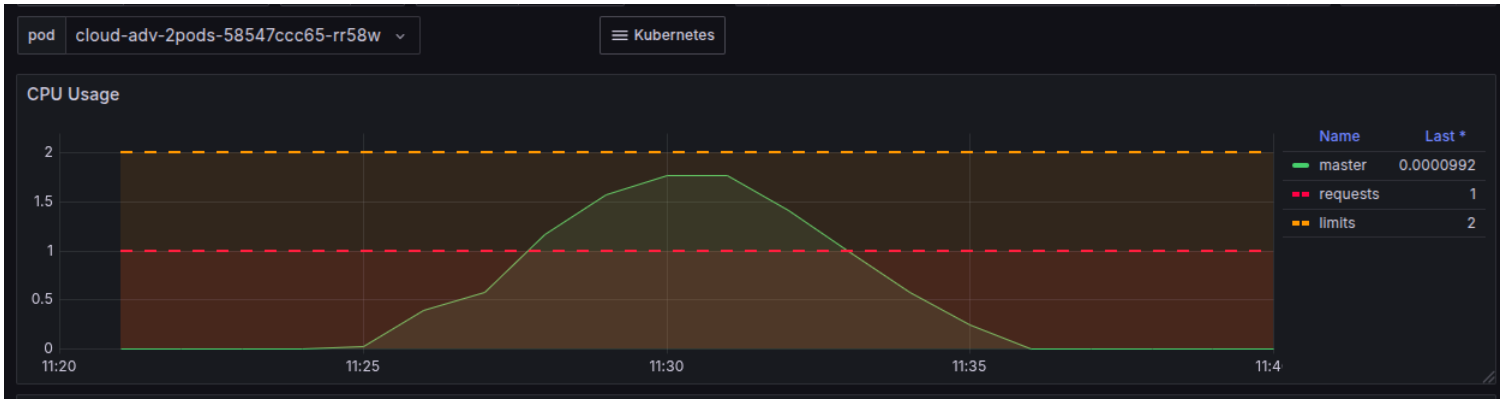


Figure 5: CPU usage pod on vm

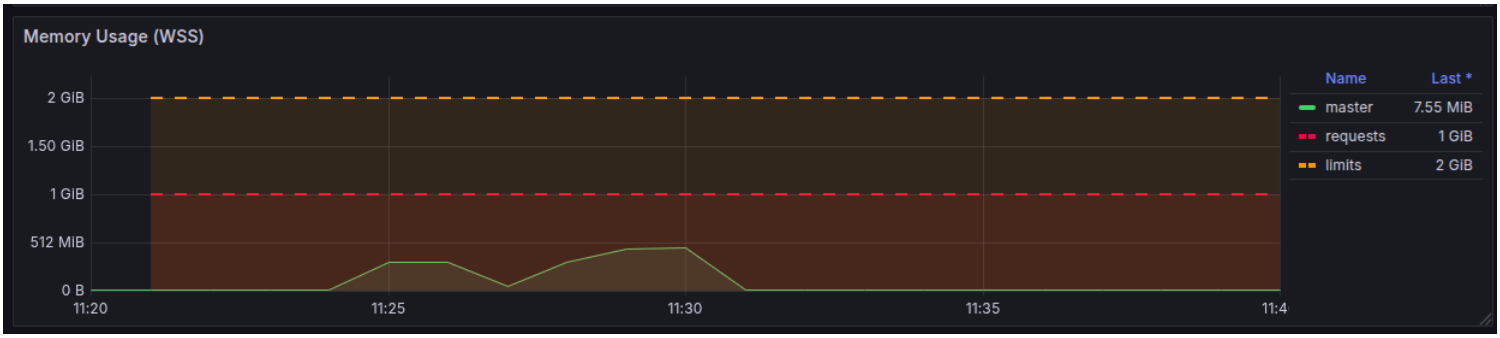


Figure 6: Memory usage pod on vm

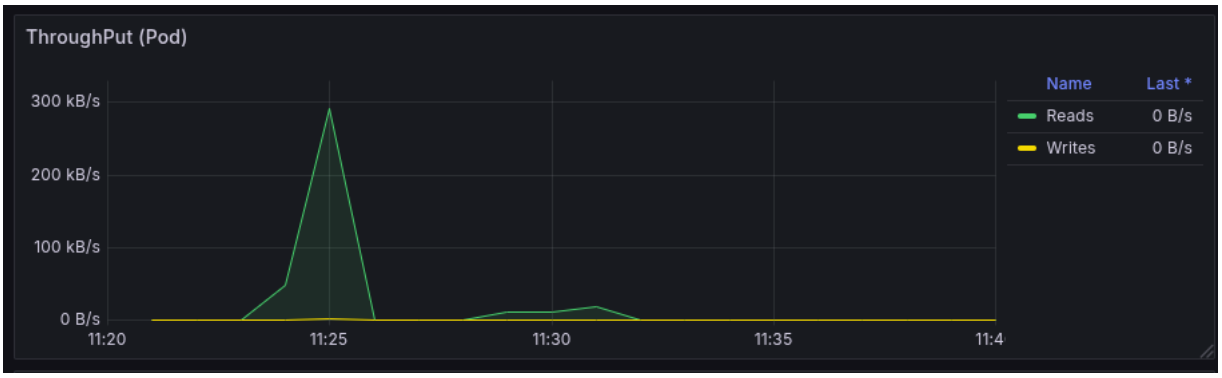


Figure 7: Network throughput pod on vm

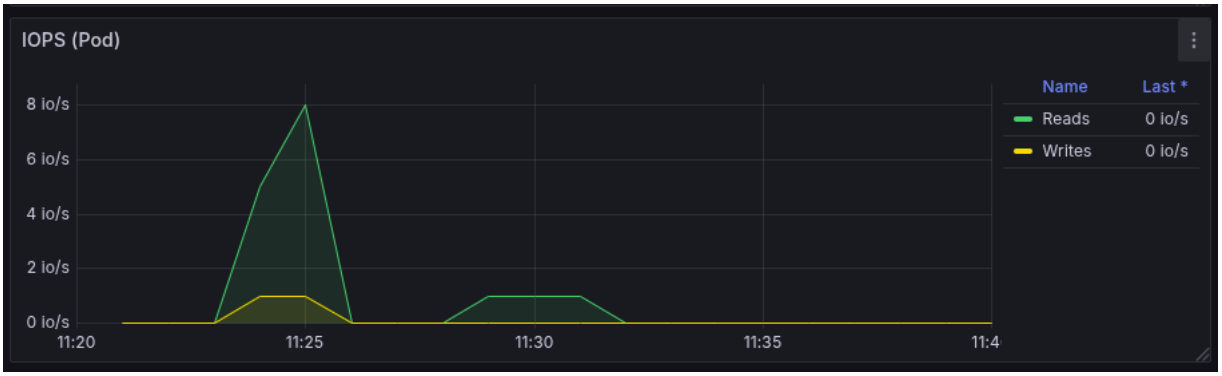


Figure 8: Disk I/O pod on vm

Here are some graphs made with Prometheus (I attach just few of them because they are basically created by the same query that Grafana has but with worse quality. They are mainly here to show that the data that Grafana scrapes exists in Prometheus). There are some differences between the data but this could be due to how Prometheus collects them or plots them.

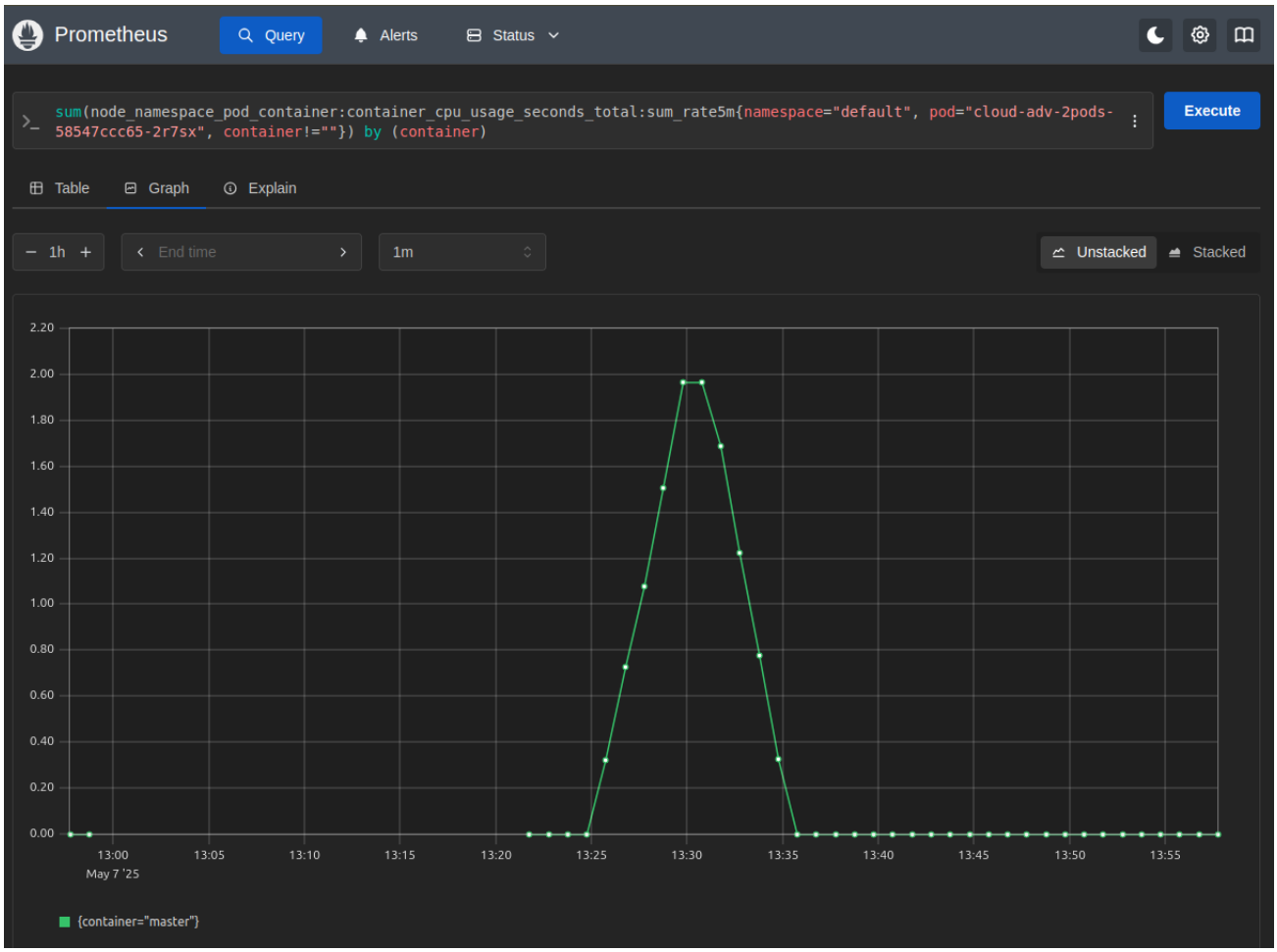


Figure 9: CPU usage pod on laptop

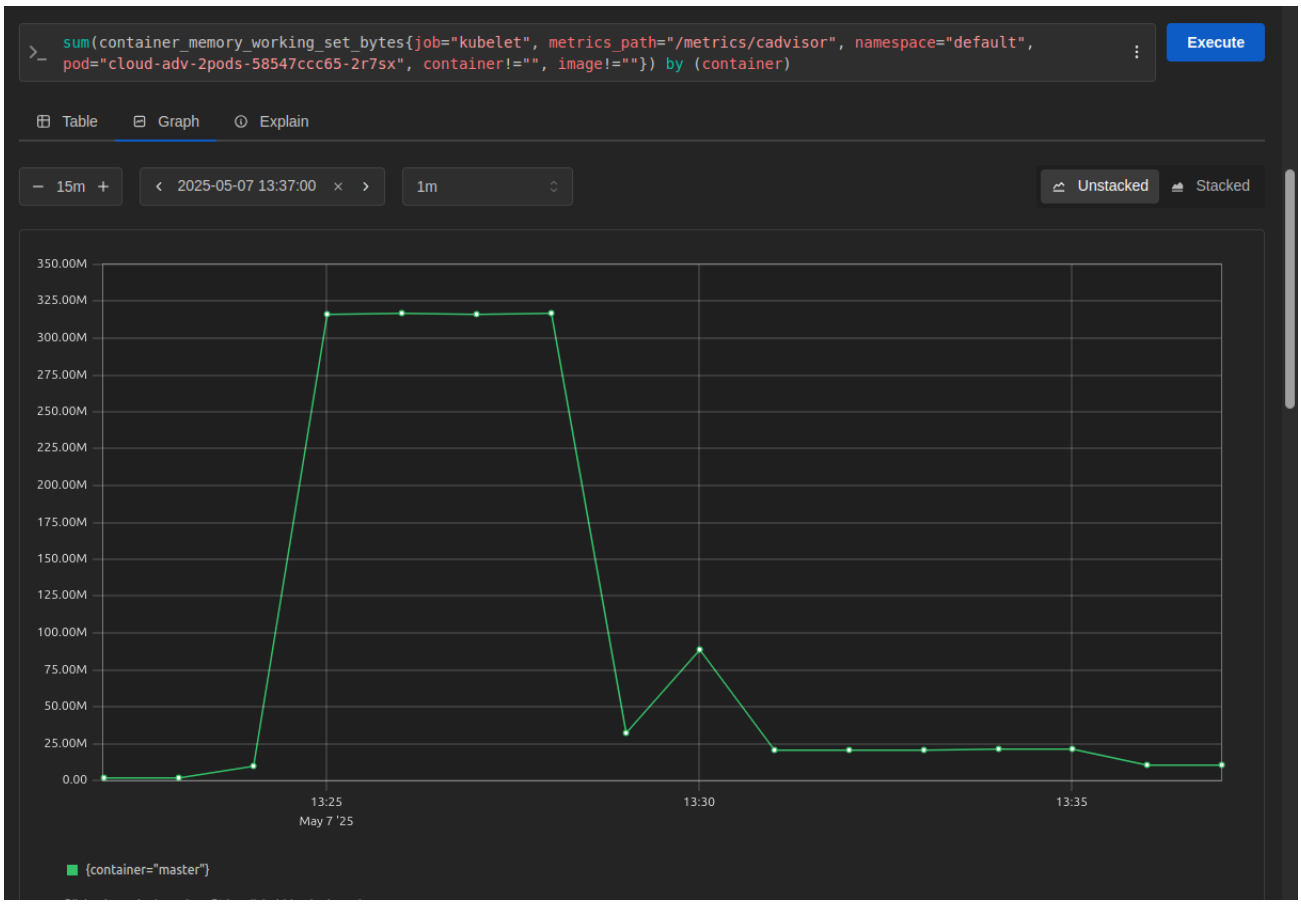


Figure 10: Memory usage pod on laptop

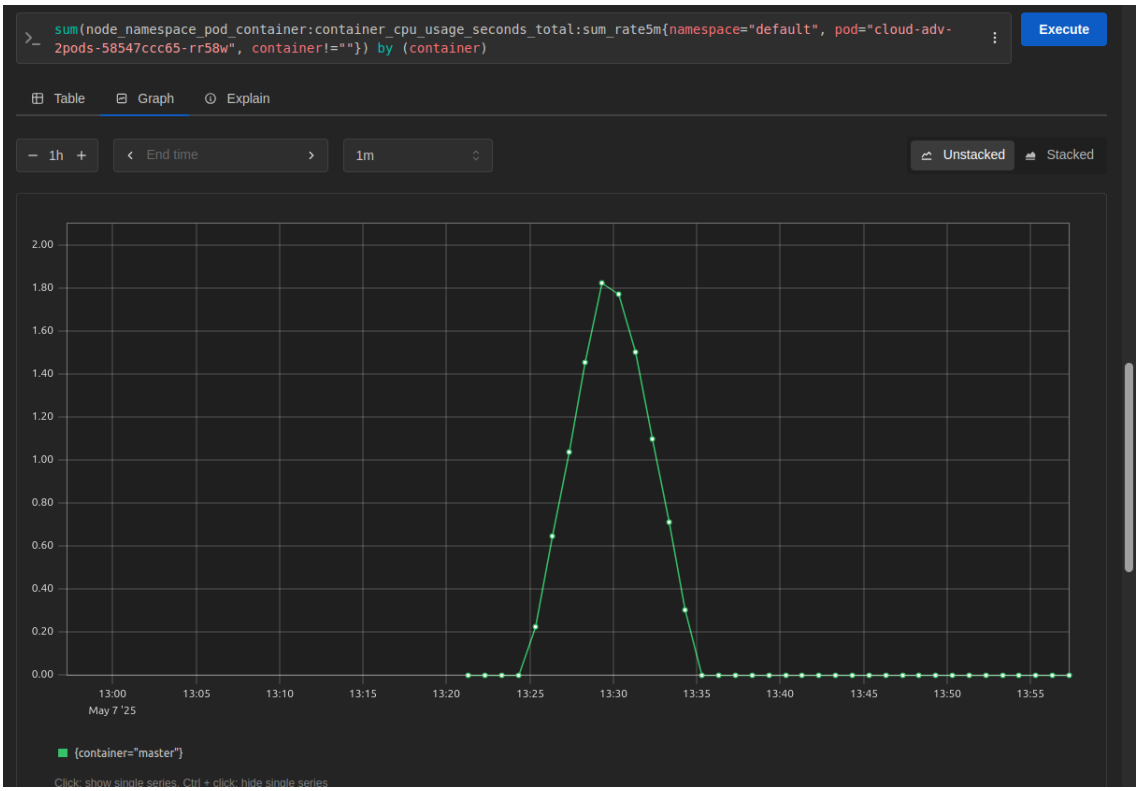


Figure 11: CPU usage pod on vm

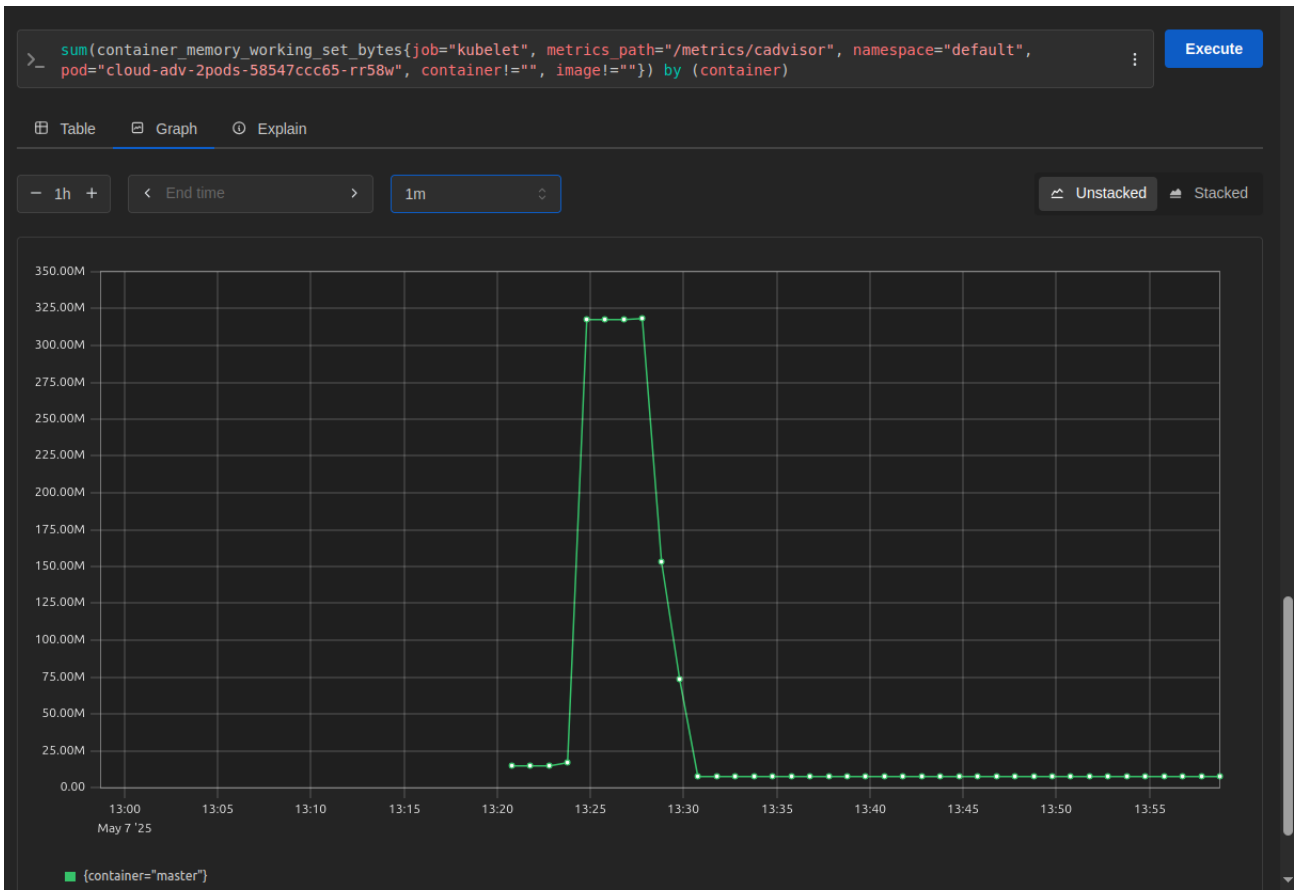


Figure 12: Memory usage pod on vm

I also had a problem with Grafana because I noticed that the graphs are slightly different from Grafana to Prometheus and probably this is due to the different elaboration of data made by Prometheus. I also noticed that in the CPU usage, the query using "irate" does not really match the output grahp in Prometheus, and this could be due to an update that they made to Grafana a couple of weeks ago. My version is the Grafana v11.5.2, in the new one they corrected this bug. Reference `https://github.com/prometheus-operator/kubeprometheus/...` To solve this I updated the git repo locally and did `kubectl apply -f manifest` inside such folder. This allowed me to have the newest version.

During the test I also noticed that there were some alerts in the alert section of Prometheus. I attach them below

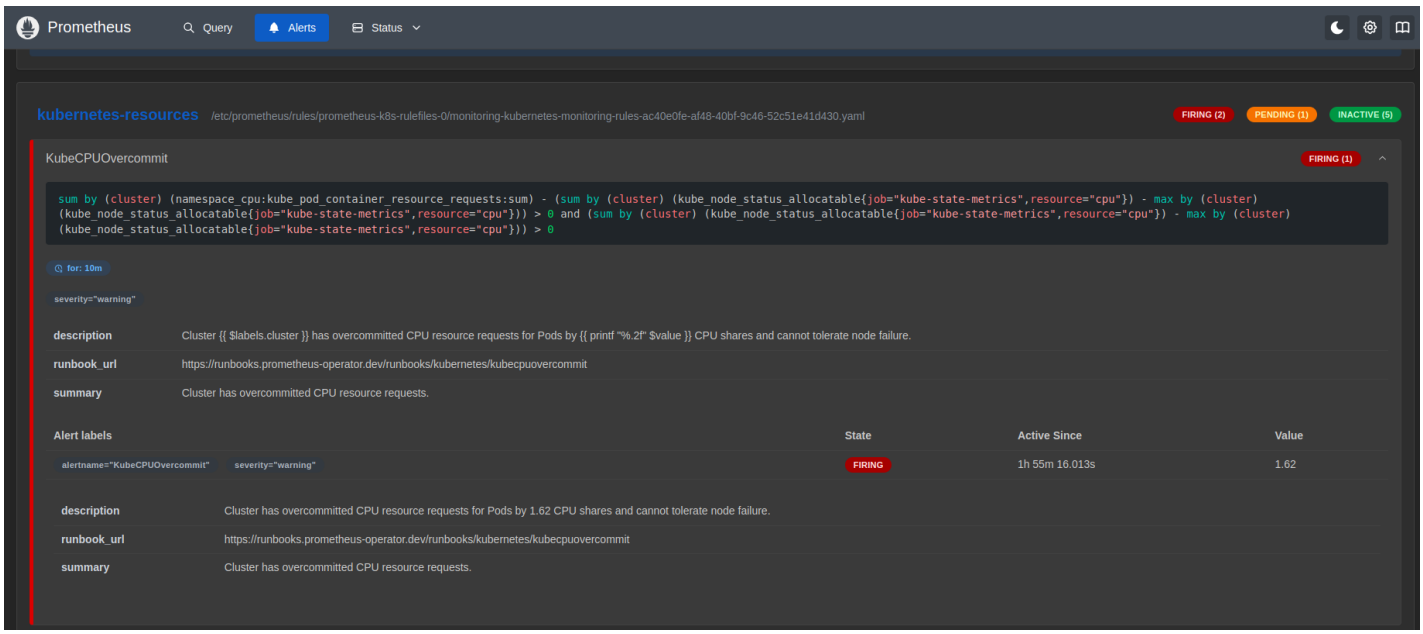


Figure 13: CPU alert

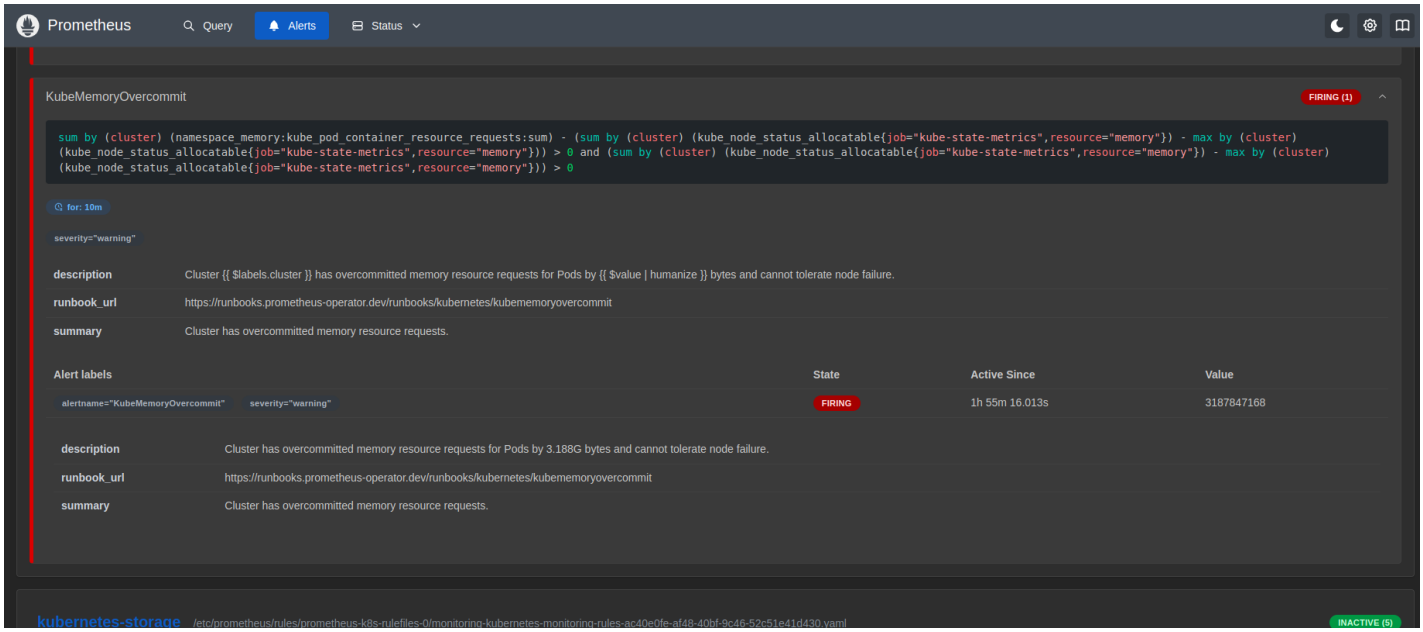


Figure 14: Memory alert

In addition to what requested, at the very beginning of my acquaintance with Kubernetes I tried to run the Kubernetes cluster on the host machine only. I started with the case of two containers in a pod but I had to solve the problem of having two containers with the same IPs (since containers in the same pod share the same IP): the only thing that distinguish them is the SSH port. So I used two services to do so and the mpirun command was fine with that, so I managed to do the test. The most successful and easier to implement case is the one with two pods with one container each: here HPCC works perfectly because we have two different IPs. I recorded the results which I write down below:

Metric	Value	Units
HPL (High-Performance Linpack)		
HPL_Tflops	0.024898	Tflops
HPL_time	26.7819	seconds
HPL_eps	1.11022e-16	
DGEMM (Double-precision General Matrix Multiply)		
StarDGEMM_Gflops	10.3763	Gflops
SingleDGEMM_Gflops	8.31044	Gflops
PTRANS (Parallel Matrix Transpose)		
PTRANS_GBs	0.631959	GB/s
PTRANS_time	0.269666	seconds
PTRANS_residual	0	
RandomAccess		
MPIRandomAccess_LCG_GUPs	0.00455124	GUPs
MPIRandomAccess_LCG_time	58.9808	seconds
MPIRandomAccess_GUPs	0.00470085	GUPs
MPIRandomAccess_time	57.1035	seconds
STREAM		
StarSTREAM_Copy	2.82227	GB/s
SingleSTREAM_Copy	12.5105	GB/s
StarSTREAM_Add	2.22314	GB/s
SingleSTREAM_Add	9.88395	GB/s
FFT (Fast Fourier Transform)		
StarFFT_Gflops	0.866078	Gflops
SingleFFT_Gflops	2.63463	Gflops
MPIFFT_Gflops	2.08734	Gflops
MPIFFT_maxErr	1.91191e-15	

As we can see the results are slightly better in some tests or slighly worse than the ones obtained with the cluster with the host and a vm. This could be due to a better communication on a host only deployment instead of a host-vm one. But in the end they are pretty similar.

2.3 Discussion

From the disk I/O and network throughput graphs we can see that there was some communication and I/O operation during the execution of the tests. From the other graphs, we can see that the CPUs are a little bit stressed and probably we could just scale up (vertical scaling) and add more CPUs (resources) to solve it. The same goes for memory. This is also confirmed by the two alerts from Alertmanager.

3 References

https://medium.com/@tech_18484/how-to-create-network-namespace-in-linux-host-83ad56c4f46f
https://medium.com/@ibshafique/exploring-linux-network-namespaces-creating-and-...
https://github.com/cri-o/cri-o/blob/main/docs/crio.8.md
https://docs.tigera.io/calico/latest/getting-started/kubernetes/quickstart
https://kubernetes.io/docs/tasks/administer-cluster/kubelet-config-file/
https://kubernetes.io/docs/concepts/services-networking/service/
https://kubernetes.io/docs/concepts/workloads/pods/
https://github.com/cri-o/packaging/blob/main/README.md#distributions-using-deb-packages
https://blog.thomas.maurice.fr/posts/some-vxlan-stuff-so-i-dont-forget/
https://kubernetes.io/docs/tasks/tools/install-kubectl-linux/#install-using-native-...
https://stackoverflow.com/questions/57868351/kubespray-fails-with-found-multiple-...
https://medium.com/@mdshofiur/creating-a-simple-vxlan-overlay-network-using-linux-...
https://cri-o.io/#container-images
https://cri-o.io/
https://kubernetes.io/docs/home/
https://github.com/prometheus-operator/kube-prometheus/blob/main/docs/access-ui.md
https://prometheus.io/docs/introduction/overview/
https://www.baeldung.com/ops/kubernetes-pull-image-again

https://runbooks.prometheus-operator.dev/
https://www.virtualbox.org/manual/ch06.html